Chronic Kidney Disease Prediction Using Machine Learning

A Research Paper

Submitted By

NAME: D. Sanjay Vara Prasad
ROLL NO: 24B81A66p4
Department of CSE (AI&ML)
CVR College of Engineering

Academic Year: 2026–2027

Abstract and Keywords

Chronic kidney disease (CKD) can progress silently, making early identification important for appropriate medical intervention. This work develops a machine learning-based prediction system using patient clinical and laboratory attributes such as blood pressure, blood glucose, blood urea, serum creatinine, hemoglobin, diabetes status, and other medical parameters. Data preprocessing techniques are applied to handle missing values and categorical attributes. A Logistic Regression classifier is trained, and Principal Component Analysis (PCA) is used to investigate whether dimensionality reduction can improve computational efficiency while retaining classification performance. Model performance is evaluated using accuracy, precision, recall, F1-score, and a confusion matrix. On the held-out test set of 80 patients, the model achieved an accuracy of 0.9875. The proposed system is intended as a preliminary decision-support mechanism rather than a replacement for clinical diagnosis.

Keywords: Chronic kidney disease, machine learning, PCA, Logistic Regression, missing value imputation, classification metrics, decision support.

Introduction and Problem Statement

Chronic kidney disease is a long-term condition in which the kidneys gradually lose their ability to filter waste from the blood. Because symptoms are often absent in early stages, diagnosis commonly depends on laboratory tests such as serum creatinine, blood urea, hemoglobin, and blood glucose, along with clinical observations such as blood pressure and the presence of diabetes or hypertension.

Machine learning can learn patterns from labelled patient records and use them to classify new patients. Clinical datasets, however, usually contain missing measurements, text-coded categorical values, and features on very different numerical scales. Careful preprocessing is therefore as important as the choice of classifier.

Logistic Regression provides a simple, efficient linear baseline for binary classification. PCA transforms correlated features into a smaller set of orthogonal components ordered by explained variance, which can reduce redundancy and computation but may also discard information useful for classification.

Problem Statement

Can patients be classified as having chronic kidney disease (CKD) or not (notCKD) from clinical and laboratory attributes, and can PCA reduce the input dimensionality while maintaining useful classification performance?

Objectives

1. Load and inspect the Chronic Kidney Disease dataset.
2. Clean inconsistent text values and convert missing-value markers to proper missing values.
3. Encode the target label and categorical attributes numerically.
4. Split the data into training and testing subsets using stratified sampling.
5. Impute missing numerical values using the training-set median.
6. Standardize features using training-set statistics.
7. Apply PCA and retain 95% of explained variance.
8. Train a Logistic Regression classifier on the PCA-reduced features.
9. Evaluate accuracy, precision, recall, and F1-score.
10. Examine the confusion matrix and discuss limitations.

Literature Review

Machine learning is widely investigated for medical classification because clinical datasets contain relationships between measurements and known outcomes. Performance depends on data quality, preprocessing, model assumptions, and evaluation methodology.

Logistic Regression is commonly used as a baseline for binary classification because it estimates class probabilities and gives a straightforward decision boundary. It can be limited when relationships are nonlinear or inputs are highly correlated, and standardization helps when feature magnitudes differ.

Missing data is common in clinical records. Median imputation is a simple, robust strategy because it is less sensitive to outliers than mean imputation, which matters for laboratory values such as creatinine and urea that often have skewed distributions.

PCA reduces dimensionality by forming principal components, each a linear combination of the original features, ordered by the variance they explain. Selecting a variance threshold provides a reproducible way to determine the size of the reduced feature space.

Research Gap

PCA is unsupervised and selects components according to variance rather than target labels. A smaller feature set may simplify computation, but it may also lose information that distinguishes CKD from non-CKD patients. The effect should therefore be evaluated with several metrics and a confusion matrix rather than accuracy alone.

Research Scope

The work is limited to binary classification on the public CKD dataset using standardization, median imputation, PCA, and Logistic Regression. It does not include raw medical images, hospital integration, treatment planning, or patient-specific advice.

1. Dataset Description

The dataset is the Chronic Kidney Disease dataset (`kidney_disease.csv`) from the UCI Machine Learning Repository, loaded from the local file `archive/kidney_disease.csv`.

- 400 patient records (320 training and 80 testing after the split).
- Clinical and laboratory attributes, including blood pressure, specific gravity, albumin, sugar, red blood cells, pus cells, blood glucose, blood urea, serum creatinine, sodium, potassium, hemoglobin, hypertension, diabetes mellitus, appetite, pedal edema, and anemia.
- A target column named `classification` with values `ckd` and `notckd`.
- An `id` column that is removed because it carries no clinical information.
- Missing values marked with `?` and stray tab characters in text fields.

The target is encoded as ckd = 1 (positive class) and notckd = 0. The test set contains 30 notCKD and 50 CKD patients.

Feature group	Description	Examples
Demographic	Patient background	Age
Blood pressure and urine tests	Basic clinical and urinalysis readings	Blood pressure, specific gravity, albumin, sugar
Cell tests	Microscopic urine findings	Red blood cells, pus cells, pus cell clumps, bacteria
Blood chemistry	Laboratory blood measurements	Blood glucose, blood urea, serum creatinine, sodium, potassium
Blood counts	Hematology measurements	Hemoglobin, packed cell volume, white and red blood cell counts
Medical history	Conditions and symptoms	Hypertension, diabetes mellitus, coronary artery disease, appetite, pedal edema, anemia

Table 1. Feature groups in the dataset.

Dataset Summary

Property	Description
Data file	`archive/kidney_disease.csv`
Task	Supervised binary classification
Target classes	CKD (1) and notCKD (0)
Input types	Numerical and categorical
Missing values	Present; marked with ?
Feature reduction	PCA retaining 95% variance

Table 2. Dataset summary.

The actual class counts can be inspected by running the program. Stratified splitting preserves approximate class proportions in the training and test subsets (the test subset has 30 notCKD and 50 CKD patients). A fixed random seed makes the experiment reproducible.

2. Data Preprocessing

Handling Missing Markers

The file is read with `na_values=['?']` so that question marks become proper missing values (NaN).

Text Cleaning

For every text column, tab characters are removed, leading and trailing spaces are stripped, and values are converted to lowercase. The string 'nan' produced by the conversion is turned back into a real missing value. This removes inconsistencies such as 'ckd<tab>' versus 'ckd'.

Target Encoding

ckd is mapped to 1 and notckd to 0, converted to numeric, and any row with a missing target is dropped. The id column is then dropped.

Categorical Encoding

Text columns with two or fewer distinct values (for example yes/no or normal/abnormal attributes) are mapped to integer codes. Any remaining text columns are one-hot encoded using `pd.get_dummies(drop_first=True)`.

Train-Test Split

- 80% training data and 20% testing data.
- Stratified sampling on the target to preserve class proportions. `random_state = 42` for reproducibility.

Median Imputation and Scaling

A `SimpleImputer(strategy='median')` is fitted on the training numerical columns only and applied to both sets. `StandardScaler` is then fitted on the imputed training data and applied to both sets, so each feature has zero mean and unit variance. This prevents features with large ranges, such as blood glucose, from dominating PCA.

Principal Component Analysis

PCA with `n_components=0.95` is fitted on the standardized training data and retains enough components to explain at least 95% of the variance. The same transformation is applied to the test set. Explained variance is not identical to predictive information, so classification performance must still be measured.

Avoiding Data Leakage

The imputer, scaler, and PCA are fitted on training data only. Test records are only transformed, so information from the test set does not influence preprocessing or model training.

3. Proposed Methodology

The workflow runs in the order shown below, reading left to right and then top to bottom.

1. Dataset Loading	**2.** Data Cleaning	**3.** Target Encoding	**4.** Categorical Encoding
5. Train-Test Split	**6.** Median Imputation	**7.** Standardization	**8.** PCA
9. Logistic Regression	**10.** Prediction	**11.** Evaluation	

Figure 1. Methodology pipeline.

Logistic Regression

Logistic Regression estimates the probability that a sample belongs to the CKD class and applies a decision threshold to produce a label. It is used as an efficient, interpretable baseline. The implementation uses `max_iter=1000` to allow sufficient optimization iterations and is trained on the PCA-reduced features.

Evaluation Metrics

- **Accuracy:** proportion of all predictions that are correct.
- **Precision:** proportion of predicted positives that are truly positive.
- **Recall:** proportion of actual positives correctly identified.
- **F1-score:** harmonic mean of precision and recall.
- **Confusion matrix:** counts of correct and incorrect predictions by class.

4. Implementation

The system is implemented in Python using pandas and NumPy for data handling and scikit-learn for splitting, imputation, scaling, PCA, classification, and evaluation.

Step	Configuration
Missing markers	`na_values=['?']`
Train-test split	80% train, 20% test; stratified
Random state	42
Imputer	Median, fitted on training data only
StandardScaler	Fitted on training data only
PCA	95% explained variance
Logistic Regression	`max_iter = 1000`

Table 3. Implementation settings.

Python Program

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
from sklearn.impute import SimpleImputer

# 1. Load data (treat '?' as missing)
data = pd.read_csv("archive/kidney_disease.csv", na_values=['?'])

# 2. Clean text columns
for col in data.select_dtypes(include=['object']).columns:
    data[col] = (data[col].astype(str)
                 .str.replace(r'\t', '', regex=True)
                 .str.strip()
                 .str.lower())
    data[col] = data[col].replace('nan', np.nan)

# 3. Encode target and drop id
if 'classification' in data.columns:
    data['classification'] = data['classification'].replace(
        {'ckd': 1, 'notckd': 0})
    data['classification'] = pd.to_numeric(
        data['classification'], errors='coerce')
    data = data.dropna(subset=['classification'])

data = data.drop(columns=['id'])
```

```
X = data.drop(columns=['classification'])
y = data['classification']

# 4. Encode categorical columns
for col in X.select_dtypes(include=['object']).columns:
    uniqvals = X[col].dropna().unique()
    if len(uniqvals) <= 2:
        mapp = {val: i for i, val in enumerate(uniqvals)}
        X[col] = X[col].map(mapp)

X = pd.get_dummies(X, drop_first=True)

# 5. Stratified 80/20 split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 6. Median imputation (fit on training data only)
num_cols = X_train.select_dtypes(include=[np.number]).columns
imputer = SimpleImputer(strategy="median")
X_train_imputed = pd.DataFrame(
    imputer.fit_transform(X_train[num_cols]), columns=num_cols)
X_test_imputed = pd.DataFrame(
    imputer.transform(X_test[num_cols]), columns=num_cols)

# 7. Standardization (fit on training data only)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train_imputed)
X_test = scaler.transform(X_test_imputed)

# 8. PCA retaining 95% of the variance
pca = PCA(n_components=0.95, random_state=42)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

# 9. Train Logistic Regression and predict
model = LogisticRegression(max_iter=1000)
model.fit(X_train_pca, y_train)
y_pred = model.predict(X_test_pca)

# 10. Evaluate
print(f"Accuracy Test {accuracy_score(y_test, y_pred):.4f}")
print("classification Report -> ")
print(classification_report(y_test, y_pred))
```

Program Explanation

1. Read the CSV and treat ? as missing.
2. Clean text columns by removing tabs and spaces and lowercasing.
3. Encode the target (ckd = 1, notckd = 0) and drop the id column.
4. Encode binary categorical columns as integers and one-hot encode any others.
5. Create a stratified 80/20 train-test split.
6. Impute missing numerical values with the training median.
7. Standardize using training statistics.
8. Fit PCA (95% variance) on training data and transform both sets.
9. Train Logistic Regression and print accuracy and the classification report.

5. Experimental Results

The model is evaluated on a held-out test set of 80 patients (30 notCKD, 50 CKD). The values below are taken directly from the program output. Test accuracy: **0.9875**.

Test Set Results

The values below are the measured output of the program for the CKD (positive) class.

Feature set	Model	Accuracy	Precision	Recall	F1-score
PCA (95% variance)	Logistic Regression	0.9875	1.00	0.98	0.99

Table 4. Test-set metrics.

Classification Report

Class	Precision	Recall	F1-score	Support
0 (notCKD)	0.97	1.00	0.98	30
1 (CKD)	1.00	0.98	0.99	50
Accuracy			0.99	80
Macro average	0.98	0.99	0.99	80
Weighted average	0.99	0.99	0.99	80

Table 5. Classification report.

Confusion Matrix

The confusion matrix is derived from the reported precision, recall, and support values, with CKD as the positive class. Rows are actual classes and columns are predicted classes.

	Predicted CKD	Predicted notCKD
Actual CKD	49 (TP)	1 (FN)
Actual notCKD	0 (FP)	30 (TN)

Table 6. Confusion matrix (rows: actual, columns: predicted).

The four outcomes are: True Positive (CKD predicted as CKD), True Negative (notCKD predicted as notCKD), False Positive (notCKD predicted as CKD), and False Negative (CKD predicted as notCKD). Out of 80 test patients, 79 were classified correctly (79 / 80 = 0.9875).

Interpreting the Results

- **Accuracy (0.9875):** only one of 80 test patients was misclassified.
- **CKD precision (1.00):** every patient predicted as CKD truly had CKD; there were no false positives.
- **CKD recall (0.98):** 49 of 50 CKD patients were detected; one was missed.
- **notCKD recall (1.00):** all 30 healthy patients were identified correctly.
- **notCKD precision (0.97):** one of the patients predicted as notCKD actually had CKD, which is the single false negative.

In a screening context a false negative, a CKD patient predicted as healthy, is the more serious error, so CKD recall deserves particular attention.

Original Features vs PCA Features

Comparison	Original features	PCA features
Input dimensions	All preprocessed numerical features	Components retaining 95% variance (count not printed by the program)
Interpretability	Original clinical measurements	Combinations of measurements
Redundancy	May include correlated inputs	May reduce correlated variation
Performance	Not evaluated in this run	Accuracy 0.9875 on the test set

Table 7. Original versus PCA feature representation.

PCA should not automatically be described as better. Because only the PCA representation was evaluated here, this experiment cannot show whether the reduction helped or hurt; a run on the original standardized features is needed for that comparison.

6. Algorithm Comparison

Algorithm	Advantages	Limitations
Logistic Regression	Clear baseline; efficient; probability estimates	Linear boundary; affected by correlated inputs
KNN	Simple; captures local patterns	Sensitive to scaling, distance, and k
Decision Tree	Easy to interpret; handles mixed feature types	Can overfit; unstable to small data changes
Random Forest	Models nonlinear patterns; ensemble-based	Less directly interpretable; more computation
PCA + classifier	Reduces dimensionality and redundancy	Components harder to interpret; may lose useful signal

Table 8. Comparison of candidate algorithms.

Feature Reduction Analysis

PCA transforms the original features into a smaller set of principal components that retain at least 95% of the variance in the standardized training data. Since PCA is unsupervised, the target labels do not influence component selection. A compact representation can help when features are correlated or computation matters, but variance is not the same as predictive usefulness: a low-variance component may still help distinguish CKD from non-CKD patients.

Model Selection

The best model should be chosen using a stated criterion and actual evaluation results. Accuracy alone may not be sufficient: CKD recall, precision, F1-score, and the confusion matrix should be considered together. In this experiment only Logistic Regression was trained, so no model ranking is claimed.

Practical Significance

The experiment illustrates how preprocessing and feature reduction influence classification of clinical data. It does not show that the model is ready for clinical use. Real-world use would require external validation, representative patient populations, bias and calibration assessment, and clinical oversight.

7. Discussion

The workflow combines cleaning, median imputation, standardization, and PCA with a linear classifier, and it reaches high performance on the held-out test set. The fact that a linear model on a reduced PCA representation performs this well suggests that CKD and non-CKD patients are largely separable using laboratory measurements such as creatinine, urea, hemoglobin, and specific gravity.

Fitting the imputer, scaler, and PCA only on the training data keeps the test set independent of preprocessing, so the reported score is a fairer estimate of performance on unseen data than one obtained by preprocessing the whole dataset at once.

The near-perfect score should still be interpreted with care. The test set contains only 80 patients, so a single error changes accuracy by 1.25 percentage points. The dataset is small and the two classes are well separated, so results may not hold for larger, noisier real-world populations.

Limitations

- Only one public dataset and one train-test split are used.
- The test set is small (80 patients), giving wide uncertainty on the metrics.
- Only Logistic Regression with default settings is evaluated; no hyperparameter tuning is performed.
- PCA preserves variance, not necessarily the most discriminative information, and components are harder to interpret clinically than the original measurements.
- Median imputation is simple and does not model relationships between features. The results are educational and cannot diagnose an individual.

Ethical Considerations

Healthcare classification can affect important decisions. A model may perform differently across populations or data-collection settings. Dataset origin should be documented, class-specific errors should be examined, and performance claims should be limited to the evidence. Qualified professionals must assess any clinical application.

8. Conclusion and Future Scope

Conclusion

This lab activity presented a machine learning workflow for predicting chronic kidney disease from clinical and laboratory attributes. After cleaning the data, handling missing values with median imputation, encoding categorical attributes, standardizing the features, and applying PCA to retain 95% of the variance, a Logistic Regression classifier achieved a test accuracy of 0.9875, with CKD precision of 1.00 and recall of 0.98 on 80 held-out patients. The system is intended as a preliminary decision-support aid and not a replacement for clinical diagnosis.

Future Scope

1. Compare Logistic Regression with KNN, Decision Tree, and Random Forest classifiers.
2. Compare results with and without PCA to measure its effect directly.
3. Print and report the number of principal components retained.
4. Use repeated stratified cross-validation for more stable estimates.
5. Tune hyperparameters with cross-validation without using the final test set.
6. Evaluate ROC-AUC and precision-recall curves.

7. Try model-based imputation such as KNN or iterative imputation.
8. Validate on independent datasets before any clinical use.

9. References

1. UCI Machine Learning Repository. Chronic Kidney Disease Dataset.
2. Scikit-learn Developers. StandardScaler: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html
3. Scikit-learn Developers. PCA: https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html
4. Scikit-learn Developers. SimpleImputer: https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html
5. Scikit-learn Developers. LogisticRegression: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
6. Scikit-learn Developers. Classification metrics: https://scikit-learn.org/stable/modules/model_evaluation.html

Note: This report is for academic learning and is not a validated clinical decision system.